JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang

Interface CharSequence

All Known Subinterfaces:

Name

All Known Implementing Classes:

CharBuffer, Segment, String, StringBuffer, StringBuilder

```
public interface CharSequence
```

A CharSequence is a readable sequence of char values. This interface provides uniform, read-only access to many different kinds of char sequences. A char value represents a character in the *Basic Multilingual Plane (BMP)* or a surrogate. Refer to Unicode Character Representation for details.

This interface does not refine the general contracts of the equals and hashCode methods. The result of comparing two objects that implement CharSequence is therefore, in general, undefined. Each object may be implemented by a different class, and there is no guarantee that each class will be capable of testing its instances for equality with those of the other. It is therefore inappropriate to use arbitrary CharSequence instances as elements in a set or as keys in a map.

Since:

1.4

Method Summary

Abstract Methods | Default Methods

Modifier and Type	Method	Description

char	**charAt**(int index)	Returns the char value at the specified index.
default **IntStream**	**chars**()	Returns a stream of int zero-extending the char values from this sequence.
default **IntStream**	**codePoints**()	Returns a stream of code point values from this sequence.
int	**length**()	Returns the length of this character sequence.
CharSequence	**subSequence**(int start, int end)	Returns a CharSequence that is a subsequence of this sequence.
String	**toString**()	Returns a string containing the characters in this sequence in the same order as this sequence.

Method Detail

length

```
int length()
```

Returns the length of this character sequence. The length is the number of 16-bit chars in the sequence.

Returns:

the number of chars in this sequence

charAt

```
char charAt(int index)
```

Returns the char value at the specified index. An index ranges from zero to length() - 1. The first char value of the sequence is at index zero, the next at index one, and so on, as for array indexing.

If the char value specified by the index is a surrogate, the surrogate value is returned.

Parameters:

`index - the index of the char value to be returned`

Returns:

`the specified char value`

Throws:

`IndexOutOfBoundsException - if the index argument is negative or not less than length()`

subSequence

```
CharSequence subSequence(int start,
                         int end)
```

Returns a `CharSequence` that is a subsequence of this sequence. The subsequence starts with the `char` value at the specified index and ends with the `char` value at index `end - 1`. The length (in `chars`) of the returned sequence is `end - start`, so if `start == end` then an empty sequence is returned.

Parameters:

`start - the start index, inclusive`

`end - the end index, exclusive`

Returns:

`the specified subsequence`

Throws:

`IndexOutOfBoundsException - if start or end are negative, if end is greater than length(), or if start is greater than end`

toString

```
String toString()
```

Returns a string containing the characters in this sequence in the same order as this sequence. The length of the string will be the length of this sequence.

Overrides:

`toString in class Object`

Returns:

`a string consisting of exactly this sequence of characters`

chars

```
default IntStream chars()
```

Returns a stream of `int` zero-extending the `char` values from this sequence. Any char which maps to a surrogate code point is passed through uninterpreted.

If the sequence is mutated while the stream is being read, the result is undefined.

Returns:

an IntStream of char values from this sequence

Since:

1.8

codePoints

```
default IntStream codePoints()
```

Returns a stream of code point values from this sequence. Any surrogate pairs encountered in the sequence are combined as if by Character.toCodePoint and the result is passed to the stream. Any other code units, including ordinary BMP characters, unpaired surrogates, and undefined code units, are zero-extended to `int` values which are then passed to the stream.

If the sequence is mutated while the stream is being read, the result is undefined.

Returns:

an IntStream of Unicode code points from this sequence

Since:

```
1.8
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.